Campus student safety app in Python

I'm making an app about student safety on campus, I have requirements that I would like to be followed when making functions necessary for the app to function properly. I would like the app and functions to be programmed in python
-The system shall allow two forms of accounts to be made, one for students and the other for helpers(other students willing to help)
-The system shall allow users(students) to request help from helpers.
-The system shall allow the request to be made with specification about student location, expected leaving time and destination.
the system shall allow helpers to receive notification about new request and be able to respond to it.
-The system shall be able to handle multiple users and requests at once, even under heavy load.
-The system shall ensure that when a helper accepts a request, no other request is colliding at the same time.
-The system shall allow only necessary information to be available to helper after accepting the request without exposing sensitive student data.
-The system shall allow helpers to communicate with student after accepting the request without any delays.

Show less

Reading frontend design skill

Now let me build the full Python backend system based on all your requirements.

Created 7 files, ran 2 commands

28/28 tests passing. Now let me copy the files to output and present them.

Ran a command, read a file

All **28 tests pass**. Here's a breakdown of how each of your requirements maps to the code:

File structure

File	Responsibility
`models.py`	Data classes: `User`, `StudentProfile`, `HelperProfile`, `HelpRequest`, `Message`
`auth.py`	Registration, login, PBKDF2 password hashing, session tokens
`request_service.py`	Full request lifecycle with thread-safe collision detection
`notification_service.py`	Async notifications, real-time push, bidirectional messaging
`server.py`	FastAPI REST API + WebSocket endpoint
`tests.py`	28 tests covering all requirements

How each requirement is implemented

Two account types — `AccountType.STUDENT` / `AccountType.HELPER` enum enforced at every layer. Helpers can't create requests; students can't accept them.

Request with location, destination, departure time — `HelpRequest` stores all three fields. Validation rejects missing locations and past departure times.

Helper notifications — `NotificationService.broadcast_new_request()` runs in a background thread, pushing to all available helpers instantly without blocking the student's API call.

Collision detection — `_helper_has_collision()` checks a ±90-minute window around each departure time. A per-helper `threading.Lock` prevents two simultaneous accepts from both slipping through.

Heavy load / concurrency — tested with 50 simultaneous student requests and 10 helpers racing to accept the same request. A `threading.RLock` on the global request store keeps everything consistent.

Privacy — `HelpRequest.safe_view_for_helper()` has two modes: before acceptance only location/destination are visible; after acceptance departure time, notes, and the student's first name only are revealed. Phone numbers, emergency contacts, and student IDs are never included.

Real-time messaging — `send_message()` / `get_messages()` are access-controlled (only the two participants). The recipient gets an immediate `new_message` notification via their queue or WebSocket subscriber.

To run the server:

```
bash
```

```
pip install -r requirements.txt
python server.py
# API docs at http://localhost:8000/docs
```

Models
PY
Download

Auth
PY
Download

Request service
PY
Download

Notification service
PY
Download

Server
PY
Download

Tests
PY
Download

Requirements
TXT
Download

Download all